# CodeCultivation

## Sistemas de Jardín Orientados a Objetos

Resumen: Construya un ecosistema de jardín digital integral mientras descubre conceptos avanzados de Python. Cree herramientas para gestionar jardines comunitarios de manera eficiente a través de enfoques basados en datos.

Versión: 2.2

# Contenidos


I Prólogo 2

II Instrucciones de IA 3

III Introducción 5

IV Instrucciones Generales 6

Ejercicio V 0: Plantando Tu Primera Semilla 7

Ejercicio VI 1: Organizador de Datos del Jardín 9

Ejercicio VII 2: Simulador de Crecimiento de Plantas 11

Ejercicio VIII 3: Fábrica de Plantas 13

Ejercicio IX 4: Sistema de Seguridad del Jardín 15

Ejercicio X 5: Tipos de Plantas Especializados 17

Ejercicio XI 6: Plataforma de Analítica del Jardín 19

XII Entrega y Envío 21

# Capítulo I

# Prólogo

En la era digital, incluso los jardines necesitan sistemas inteligentes.

Un jardinero sabio dijo una vez: "No puedes plantar flores y luego desenterrarlas todos los días para ver cómo van las raíces". Lo mismo se aplica al código: a veces el crecimiento más bello ocurre cuando confiamos en el proceso y permitimos que nuestros programas se desarrollen de forma natural, capa por capa.

Detrás de cada jardín comunitario próspero yace una red de relaciones. Las plantas no crecen aisladas; forman comunidades donde cada especie aporta algo único al tiempo que comparten necesidades comunes. Algunas plantas protegen a otras de plagas, algunas fijan nitrógeno en el suelo y otras proporcionan sombra para plántulas delicadas. Esta red interconectada de apoyo mutuo refleja cómo funcionan los sistemas de software bien diseñados—componentes individuales con roles distintos, trabajando juntos de forma armoniosa.

Así como un maestro jardinero sabe que un suelo saludable produce plantas sanas, los programadores experimentados entienden que un código bien estructurado crece hasta convertirse en aplicaciones robustas. Descubrirás que organizar tu código es como planificar un jardín: algunos elementos necesitan protección, otros espacio para crecer, y los mejores sistemas surgen cuando todo tiene su lugar y propósito adecuados.

En este proyecto, cultivarás tus habilidades de programación mientras construyes herramientas que podrían ayudar realmente a que los jardines comunitarios prosperen. Cada línea de código que escribes es una semilla plantada en el suelo digital de la posibilidad.

# Capítulo II

## Instrucciones de IA

### Contexto

Durante tu viaje de aprendizaje, la IA puede ayudar con muchas tareas diferentes. Tómate el tiempo para explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyar tu trabajo. Sin embargo, abórdalas siempre con precaución y evalúa críticamente los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca puedes estar completamente seguro de que tu pregunta estuvo bien formada o de que el contenido generado es preciso. Tus compañeros son un recurso valioso para ayudarte a evitar errores y sesgos.

### Mensaje principal

- Utiliza la IA para reducir tareas repetitivas o tediosas.
- Desarrolla habilidades para generar indicaciones  tanto de codificación como de no codificación  que beneficiarán tu futura carrera.
- Aprende cómo funcionan los sistemas de IA para anticipar y evitar riesgos comunes, sesgos y problemas éticos.
- Sigue desarrollando habilidades técnicas y de potencia trabajando con tus pares.
- Utiliza solo contenido generado por IA que entiendas plenamente y por el que puedas asumir responsabilidad.

### Reglas del aprendiz:

- Deberías tomarte el tiempo para explorar las herramientas de IA y entender cómo funcionan para que puedas usarlas de forma ética y reducir sesgos potenciales.
- Debes reflexionar sobre tu problema antes de generar indicaciones; esto te ayuda a redactar indicaciones más claras, detalladas y relevantes utilizando vocabulario preciso.
- Debes desarrollar el hábito de revisar, cuestionar y probar de forma sistemática cualquier cosa generada por IA.
- Siempre busca la revisión de pares  no te repares únicamente en tu propia validación.

## Resultados de la fase:

- Desarrollar habilidades de prompting tanto de uso general como específicas del dominio.
- Aumenta tu productividad con un uso efectivo de herramientas de IA.
- Continúa fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

# Comentarios y ejemplos:

- Con frecuencia encontrarás situaciones  exámenes, evaluaciones y más  donde debes demostrar una comprensión real. Prepárate, sigue desarrollando tus habilidades técnicas y interpersonales.
- Explicar tu razonamiento y debatir con tus compañeros a menudo revela brechas en tu comprensión. Haz del aprendizaje entre pares una prioridad.
- Las herramientas de IA a menudo carecen de tu contexto específico y tienden a ofrecer respuestas genéricas. Tus compañeros, que comparten tu entorno, pueden aportar ideas más relevantes y precisas.
- Donde la IA tiende a generar la respuesta más probable, tus compañeros pueden proporcionar perspectivas alternativas y matices valiosos. Confía en ellos como un punto de control de calidad.

**✓ Buena práctica:**

> Le pregunto a la IA: “¿Cómo pruebo una función de ordenación?” Me da algunas ideas. Las intento y reviso los resultados con un compañero. Refinamos el enfoque juntos.

**✗ Mala práctica:**

> Pido a la IA que escriba una función completa, la copio y la pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad y fracaso mi proyecto.

**✓ Buena práctica:**

> Uso la IA para ayudar a diseñar un analizador. Luego recorro la lógica con un compañero. Detectamos dos errores y lo reescribimos juntos: mejor, más limpio y completamente entendido.

**✗ Mala práctica:**

> Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no logro justificarlo y fracaso en mi proyecto.

# Capítulo III

# Introducción

¡Bienvenido a Code Cultivation!

Basándose en tus fundamentos de Python de la primera actividad, ahora enfrentarás desafíos de programación más complejos mediante la creación de un sistema integral de gestión de datos de un jardín. Este proyecto introduce conceptos avanzados que hacen de Python una herramienta poderosa para modelar sistemas del mundo real.

Trabajarás en:

- Comprender cómo se estructuran y ejecutan los programas en Python
- Organizar estructuras de datos complejas de manera eficiente
- Creando componentes de código reutilizables
- Construyendo sistemas que pueden adaptarse y ampliarse
- Protegiendo la integridad de datos en entornos colaborativos
- Diseñando arquitecturas de software escalables

Cada ejercicio se basa en los anteriores, creando al final un ecosistema digital de jardín completo.

IMPORTANTE: Este módulo comienza con la estructura básica de un programa de Python, y luego progresa a la Programación Orientada a Objetos. Cada ejercicio debe contener las definiciones solicitadas y cualquier código requerido. Puedes incluir código de prueba simple al final de cada archivo usando bloques if __name__ == "__main__": para tus propias pruebas.

# Capítulo IV
# Instrucciones generales

- Tu código debe estar escrito en Python 3.10+
- Tu código debe respetar los estándares del linter flake8
- Cada ejercicio debe estar en su propio archivo
- Usa convicciones de nomenclatura adecuadas: clases en PascalCase, funciones y variables en snake_case
- Todas las funciones y métodos deben incluir hints de tipo
- No necesitas manejar la validación de entrada a menos que se mencione explícitamente
- Enfócate en demostrar con claridad conceptos de programación
- Tus programas deben ejecutarse siempre sin errores

**Nota sobre el Ecosistema del Jardín Digital:** Este proyecto se centra en Python conceptos de programación, comenzando desde la estructura básica del programa y progresando hacia la Programación Orientada a Objetos. Cada ejercicio introduce nuevas características mientras se construye un sistema de gestión de jardines cohesivo. Los ejercicios posteriores reutilizarán conceptos de los anteriores.

**Programación Orientada a Objetos:** A partir del Ejercicio 1, todos los ejercicios en este módulo requieren el uso de clases. Palabras clave de Python como 'class' y 'def' son características fundamentales del lenguaje y no necesitan ser enumeradas en las funciones autorizadas para cada ejercicio.

# Capítulo V Ejercicio 0: Plantando tu primera semilla

| | |
|---|---|
| | Ejercicio0 |
| ft_garden_intro | |
| Directorio: ex0/ | |
| Archivos a entregar: ft_garden_intro.py | |
| Autorizado: print() | |

Antes de que podamos construir sistemas complejos de gestión de jardines, necesitas entender cómo funcionan los programas en Python.

Todo programa en Python necesita un punto de inicio, un lugar donde comienza la ejecución. ¡Al igual que plantar tu primera semilla en un jardín, aquí es donde comienza tu viaje en la programación!

Tu tarea es crear tu primer programa en Python que muestre información sobre una planta en tu jardín.

Requisitos:

- Crea un programa que se ejecute cuando se ejecute directamente
- Utiliza el patrón especial if __name__ == "__main__":
- Guarda la información de la planta en variables simples (nombre, altura, edad)
- Muestra la información de la planta usando print()

Ejemplo:

```
$> python3 ft_garden_intro.py
=== Bienvenido a Mi Jardín
=== Planta: Rosa Altura: 25cm
Edad: 30 días


=== Fin del Programa ===
```

Esta es tu primera semilla plantada en el jardín de la programación Python. Entender cómo comienzan y ejecutan los programas es fundamental antes de avanzar a conceptos más complejos como funciones y clases.

**Nota:** A partir de este ejercicio y para todos los ejercicios siguientes en este módulo (y proyectos futuros de Python), se te permite usar if __name__ == "__main__": bloques para probar tu código. Este patrón no se repetirá en la lista de funciones autorizadas de ejercicios subsecuentes, pero permanece disponible a lo largo.

¿Qué sucede si eliminas la línea if __name__ == "__main__"? Pruébalo y observa la diferencia. Además, ¿has notado que algunos archivos de Python comienzan con una línea especial que empieza con #!? Investiga qué hace esta línea de "shebang" y por qué podría ser útil para hacer que tus scripts sean ejecutables directamente.

# Capítulo VI

## Ejercicio 1: Organizador de Datos del Jardín

| | |
|---|---|
| | Ejercicio1 |
| ft_garden_data | |
| Directorio: ex1/ | |
| Archivos para enviar: ft_garden_data.py | |
| Autorizado: print(), range() | |

El jardín comunitario necesita hacer un seguimiento de varias plantas con su información. Debes almacenar y mostrar datos de varias plantas de manera eficiente.

Cada planta tiene:

- Un nombre
- Altura en centímetros
- Edad en días

Crea un programa que gestione los datos de al menos 3 plantas diferentes y muestre su información de forma organizada.

Debes crear una clase Plant que sirva como plano para cualquier planta, en lugar de manejar cada una individualmente. Por ejemplo, cada planta podría tener un nombre, una altura y una edad, pero necesitarás una forma de organizar estos datos que tenga sentido.

.

Requerido: Crear un plano Plant que pueda representar cualquier planta con sus atributos.

Ejemplo:

```
$> python3 ft_garden_data.py ===
Registro de Plantas del Jardín ===
Rosa: 25cm, 30 días; Sol: 80cm, 45
días; Cactus: 15cm, 120 días
```

¿Cómo está almacenando actualmente la información de las plantas? ¿Qué desafíos podrían surgir con más plantas?

# Capítulo VII

## Ejercicio 2: Simulador de crecimiento de plantas

| | |
|---|---|
| | Ejercicio2 |
| ft_crecimiento_planta | |
| Directorio: ex2/ | |
| Archivos para enviar: ft_plant_growth.py | |
| Autorizado: print(), range() | |

El administrador del jardín quiere simular el crecimiento de las plantas a lo largo del tiempo. Debes rastrear cómo cambian las plantas y proporcionar operaciones para modificar su estado.

Requisitos:

- Reutiliza tu clase Planta del Ejercicio 1
- Las plantas deben poder crecer() y envejecer() - piensa en qué acciones puede realizar una planta
- Necesitarás una forma de obtener información() sobre el estado actual de la planta
- Simular una semana de crecimiento para varias plantas
- Considera cómo las plantas deben cambiar con el tiempo a través de sus propias acciones

Tu programa debe mostrar cómo las plantas cambian con el tiempo. Piensa en darles comportamientos que puedan realizar por sí mismas.

Ejemplo:

```
$> python3 ft_plant_growth.py ===
Día 1 === Rosa: 25 cm, 30 días de
edad === Día 7 === Rosa: 31 cm,
36 días de edad Crecimiento de esta
semana: +6 cm
```

¿Cómo estás manejando las operaciones sobre datos de plantas? ¿Hay repetición en tu código?

# Capítulo VIII

## Ejercicio 3: Fábrica de Plantas

| | |
|---|---|
| | Ejercicio3 |
| ft_fábrica_de_plantas | |
| Directorio: ex3/ | |
| Archivos a enviar: ft_fábrica_de_plantas.py | |
| Autorizado: print(), range() | |

El vivero necesita crear muchas plantas rápidamente usando tu clase Plant con diferentes valores iniciales. Debes agilizar el proceso de creación de plantas e iniciarlas correctamente.

Requisitos:

- Las plantas deben crearse con su información inicial (nombre, altura inicial, edad inicial)
- Cada planta debe estar lista para usar inmediatamente después de la construcción
- Crea al menos 5 plantas diferentes con características variables
- Muestra todas las plantas creadas en un formato organizado
- Piensa en cómo puedes agilizar el proceso de creación de plantas

Considera cómo podrías configurar las plantas con sus valores iniciales de forma eficiente. ¿Qué haría que crear muchas plantas fuera más fácil?

Ejemplo:

```
$> python3 ft_plant_factory.py ===
Salida de la Fábrica de Plantas ===
Creado: Rosa (25cm, 30 días) Creado:
Roble (200cm, 365 días) Creado: Cactus
(5cm, 90 días) Creado: Girasol (80cm, 45
días) Creado: Helecho (15cm, 120 días)


Total de plantas creadas: 5
```

¿Cómo estás inicializando actualmente tus plantas? ¿Hay una forma más eficiente de configurarlas?

# Capítulo IX

## Ejercicio 4: Sistema de Seguridad del Jardín

| | |
|---|---|
| | Ejercicio4 |
| ft_jardín_seguridad | |
| Directorio: ex4/ | |
| Archivos para enviar: ft_garden_security.py | |
| Autorizado: print(), range() | |

El gerente del jardín está preocupado por la integridad de los datos. Algunos voluntarios accidentalmente corrompieron los datos de las plantas al establecer valores inválidos (alturas negativas, edades imposibles). Necesitas crear un sistema seguro que proteja y encapsule datos sensibles.

Requisitos:

- Crear un SecurePlant que proteja sus datos de la corrupción
- Proporcionar formas controladas de modificar los datos de la planta: set_height(), set_age()
- Proporcionar formas seguras de acceder a los datos de la planta: get_height(), get_age()
- Asegurar que la altura de la planta no pueda ser negativa mediante validación
- Asegurar que la edad de la planta no pueda ser negativa mediante validación
- Imprimir mensajes de error cuando se intenten valores inválidos
- Piensa en encapsulación: proteger datos importantes del acceso directo

Considera cómo podrías crear un sistema que valide los datos antes de almacenarlos, asegurando la integridad de los datos.

Ejemplo:

```
$> python3 ft_garden_security.py ===
Sistema de Seguridad del Jardín ===
Planta creada: Rosa Altura
actualizada: 25cm [OK] Edad
actualizada: 30 días [OK]

Operación inválida intentada: altura -5cm [RECHAZADA]
Seguridad: Altura negativa rechazada

Planta actual: Rosa (25cm, 30 días)
```

¿Cómo puede proteger sus datos para evitar que se dañen accidentalmente? ¿Qué mecanismos podría implementar?

# Capítulo X

## Ejercicio 5: Tipos de Plantas Especializados

| | |
|---|---|
| | Ejercicio5 |
| ft_plant_types | |
| Directorio: ex5/ | |
| Archivos para enviar: ft_plant_types.py | |
| Autorizado: super(), print(), range() | |

El jardín ahora necesita manejar diferentes tipos de plantas: flores, árboles y verduras. Cada tipo tiene características únicas pero hereda las funciones comunes de la planta de su categoría par- ent.

Requisitos:

- Comienza con una base Plant que tenga características comunes (nombre, altura, edad)
- Crea tipos especializados: Flor, Árbol y Verdura
- Cada tipo especializado debe heredar las características básicas de la planta
- Necesidades de la Flor: atributo de color y capacidad para florecer()
- Necesidades del Árbol: diámetro del tronco y capacidad para producir_sombra()
- Verdura necesita: temporada de cosecha y valor nutricional
- Al crear plantas especializadas, llama a la configuración del padre con super().__init__()
- Crea al menos 2 instancias de cada tipo de planta
- Evita duplicar código común de plantas entre diferentes tipos especializados

Piensa en las relaciones familiares en la naturaleza, ¿cómo comparten rasgos las especies mientras tienen y características únicas?

Ejemplo:

```
$> python3 ft_plant_types.py ===
Tipos de Plantas de Jardín ===

Rosa (Flor): 25cm, 30 días, color rojo La rosa está
floreciendo hermosamente!

Roble (Árbol): 500cm, 1825 días, diámetro de 50cm El
roble proporciona 78 metros cuadrados de sombra

Tomate (Verdura): 80cm, 90 días, cosecha de verano El
tomate es rico en vitamina C
```

Nota cuánta duplicación de código podrías tener. Cada tipo de planta comparte características comunes pero tiene características únicas. Piensa en **relaciones familiares** en la naturaleza, ¿cómo comparten rasgos las especies?

¿Cómo está manejando las características comunes compartidas por todos los tipos de plantas? ¿Existe una forma de evitar repetir el mismo código creando un árbol familiar de tipos relacionados?

# Capítulo XI

# Ejercicio 6: Plataforma de Análisis de Jardines

| | |
|---|---|
| | Ejercicio6 |
| ft_garden_analytics | |
| Directorio: ex6/ | |
| Archivos a entregar: ft_garden_analytics.py | |
| Autorizado: super(), print(), range(), staticmethod(), classmethod() | |

Construya una plataforma integral de análisis de datos de jardines que procese y analice datos de jardines. Este sistema debe manejar relaciones de datos complejas y proporcionar análisis detallados utilizando componentes anidados y cadenas de herencia.

Requisitos:

- Cree un GardenManager que pueda manejar múltiples jardines
- Incluya una utilidad GardenStats dentro de su administrador para calcular estadísticas
- Construya un árbol de familia de plantas: Planta  Florante  PrizeFlower
- Incluya un método create_garden_network() que funcione en el tipo de administrador por sí mismo
- Agregue funciones utilitarias que no necesiten datos de jardines específicos
- Muestre diferentes tipos de métodos: métodos de instancia, métodos a nivel de clase y funciones utilitarias
- Cada jardín debe rastrear colecciones de plantas y estadísticas
- Utilice su ayuda de estadísticas anidadas para calcular analíticas
- Organice todo dentro de estructuras adecuadas - evite funciones globales dispersas

Considere cómo podría organizar sistemas complejos con múltiples componentes interactuando. ¿Qué ocurre cuando necesita diferentes tipos de métodos?

Ejemplo:

```
$> python3 ft_garden_analytics.py
=== Demostración del Sistema de Gestión de Jardín ===

Árbol de Roble añadido al jardín de Alice
Rosa añadida al jardín de Alice
Girasol añadido al jardín de Alice

Alice está ayudando a que todas las plantas crezcan...
El roble creció 1 cm
La rosa creció 1 cm
El girasol creció 1 cm

=== Informe del jardín de Alice ===
Plantas en el jardín:
- Árbol de Roble: 101 cm
- Rosa: 26 cm, flores rojas (en floración)
- Girasol: 51 cm, flores amarillas (en floración), Puntos de premio: 10

Plantas añadidas: 3, Crecimiento total: 3 cm
Tipos de plantas: 1 regular, 1 en flor, 1 flores con premio

Prueba de validación de altura: True
Puntuaciones del jardín - Alice: 218, Bob: 92
Total de jardines gestionados: 2
```

Este ejercicio reúne todos los patrones de programación que has aprendido a lo largo del proyecto. Serás evaluado en tu comprensión de cómo interactúan e se organizan entre sí los diferentes componentes dentro de un sistema complejo.

¿Cómo organizas sistemas complejos con múltiples componentes que interactúan? ¿Qué sucede cuando necesitas diferentes tipos de métodos que pertenecen a la clase misma en lugar de a instancias individuales?

# Capítulo XII

# Entrega y Envío

Entrega tu tarea en tu repositorio de Git como de costumbre. Solo el trabajo dentro de tu repositorio será evaluado durante la defensa. No dudes en revisar dos veces los nombres de tus archivos para asegurarte de que sean correctos.

Durante la evaluación, es posible que se te pida explicar conceptos de programación, demostrar relaciones de herencia o ampliar tu código con nueva funcionalidad. Asegúrate de entender los principios detrás de tus implementaciones.

Solo debes devolver los archivos solicitados por la asignatura de este proyecto. Enfócate en un código limpio, bien documentado que demuestre claramente los principios de programación.